

MATLAB and Python Files for Pareto Extrapolation

Alexis Akira Toda*

Revised August 16, 2026

1 Introduction

This package implements the Pareto extrapolation algorithm developed by [2]. The method augments a conventional finite-grid approximation with an analytical Pareto tail. It computes the Pareto exponent, the stationary joint distribution of an exogenous state and a size variable such as wealth, and top wealth shares. The characterization of the Pareto exponent is based on [1].

The source repository is github.com/alexisakira/Pareto-extrapolation. The software is maintained by Alexis Akira Toda and is provided without warranty. Users should verify numerical accuracy and economic assumptions for their own application. If these routines are used in research, please cite [2].

The folder contains parallel implementations:

```
PE/
|-- matlab/                MATLAB implementation and example
|-- python/
|   |-- src/pareto_extrapolation/  Python package
|   |-- examples/                 Python example
|   |-- tests/                    MATLAB-parity and edge-case tests
|   '-- pyproject.toml
|-- readme.tex
'-- readme.pdf
```

Index and probability conventions

Let $s, t \in \{1, \dots, S\}$ denote the current and next exogenous states, $j \in \{1, \dots, J\}$ a transitory shock, and $n \in \{1, \dots, N\}$ a grid index. Whenever an $S^2 \times J$ array is required, the row for (s, t) is

$$r = S(s - 1) + t$$

in one-based notation. Thus the rows are ordered as

$$(1, 1), \dots, (1, S); \quad \dots; \quad (S, 1), \dots, (S, S).$$

In the joint transition matrix, the grid index varies fastest:

$$(s, n) = (1, 1), \dots, (1, N); \dots; (S, 1), \dots, (S, N).$$

Probability transition matrices are row-stochastic. The survival probability $V(s, t)$ is conditional on the transition from s to t ; use $V = 1$ for an infinitely lived model. Numerical inputs must be real and finite.

*Department of Economics, Emory University. Email: alexis.akira.toda@emory.edu.

2 MATLAB

The MATLAB implementation is in the `matlab` folder. It uses standard MATLAB functionality, including sparse matrices, `eigs`, `discretize`, `pchip`, and `interp1`, and has been tested with MATLAB R2024b. From the PE folder, run:

```
addpath('matlab')
example
```

2.1 Exponential grid: `expGrid.m`

```
grid = expGrid(a,b,c,N)
```

The function returns a strictly increasing $1 \times N$ shifted-log grid on the closed interval $[a, b]$, with `grid(1) = a` and `grid(end) = b` exactly. The concentration parameter must satisfy

$$a < c < \frac{a + b}{2},$$

and N must be an integer no smaller than two. For odd N , c is the middle grid point; for even N , it lies between the middle points in shifted-log space. This differs from releases through 2021, which excluded a and generated points on $(a, b]$.

2.2 Pareto exponent: `getZeta.m`

```
[zeta,typeDist] = getZeta(PS,PJ,V,G,zetaBound)
```

The inputs are:

- **PS**: the $S \times S$ transition matrix of the exogenous state;
- **PJ**: conditional shock probabilities with size $1 \times J$, $S \times J$, or $S^2 \times J$. The shorter forms assume independence from (s, t) or dependence only on s ;
- **V**: a scalar or $S \times S$ matrix of survival probabilities;
- **G**: asymptotic gross growth rates with size $S \times J$ or $S^2 \times J$; and
- **zetaBound**: an optional two-element search interval, with default $[10^{-2}, 100]$.

Define

$$M_{st}(z) = \sum_{j=1}^J \Pr(j \mid s, t) G_{stj}^z,$$
$$A(z) = PS \odot V \odot M(z),$$

where $\odot$ denotes elementwise multiplication. The Pareto exponent solves

$$\rho(A(\zeta)) = 1,$$

and `typeDist` is the normalized left Perron vector of $A(\zeta)$. A root exactly on a search bound is accepted. If no root exists inside the interval, the function returns the relevant endpoint, issues a warning, and returns `typeDist = []`.

2.3 Joint transition matrix: `getQ.m`

```
[Q,piStar] = getQ(PS,PJ,V,x0,xGrid,gstjn,Gstj,zeta)
```

The first three inputs follow `getZeta.m`. The remaining inputs are:

- `x0`: the size of a newborn or reset agent. It is used when at least one entry of V is below one and must satisfy $x_1 \leq x_0 < x_N$;
- `xGrid`: a strictly increasing N -element grid with $x_N > 0$;
- `gstjn`: the law of motion evaluated on the grid. It has JN columns, with columns $N(j-1) + 1$ through Nj corresponding to shock j . It may have S rows or S^2 rows;
- `Gstj`: optional asymptotic slopes with S or S^2 rows. Pass `[]` to estimate them from the two largest grid points; and
- `zeta`: the optional Pareto exponent. Pass `[]` to call `getZeta.m` internally.

The function returns the $SN \times SN$ transition matrix Q as a sparse matrix. If requested, `piStar` is the full $SN \times 1$ stationary distribution. The marginal distribution on the size grid is

```
xDist = sum(reshape(piStar,N,S),2);
```

The function checks the row sums of Q and the stationary-distribution residual. Pareto extrapolation requires $\zeta > 1$ for a finite mean. If $\zeta \leq 1$, the function warns and returns NaN-valued outputs.

2.4 Top wealth shares: `getTopShares.m`

```
topShare = getTopShares(topProb,wGrid,wDist,zeta)
```

`topProb` is a strictly increasing vector in $[0, 1]$, `wGrid` is a strictly increasing wealth grid, and `wDist` is a probability vector on that grid. Omit `zeta`, pass `[]`, or pass zero to use upper-grid truncation; otherwise ζ must exceed one. The output has the same row or column orientation as `topProb`.

Interpolation uses the shape-preserving `pchip` method. Under Pareto extrapolation, the wealth represented by the largest grid point is valued as

$$\tilde{w}_N = \frac{\zeta}{\zeta - 1} w_N.$$

Negative wealth is allowed, but aggregate net wealth—including the Pareto correction when applicable—must be strictly positive. Top shares can legitimately exceed one when households outside the selected group have debt, and the top-share curve can decline as indebted households enter the group. The function therefore does not clamp the result to $[0, 1]$ or force monotonicity.

2.5 Example: `example.m`

The example uses a two-state return process and a two-point transitory shock:

```
xGrid = expGrid(xMin,xMax,x0,N);
gstjn = kron(Gstj,xGrid);

[zeta,typeDist] = getZeta(PS,PJ,V,Gstj,[0.1 10]);
[Q,piStar] = getQ(PS,PJ,V,x0,xGrid,gstjn,Gstj,zeta);

xDist = sum(reshape(piStar,N,S),2);
topShare = getTopShares([0.001 0.01 0.1],xGrid,xDist,zeta);
```

It produces $\zeta \approx 1.721899$ and top 0.1%, 1%, and 10% shares of approximately 0.073619, 0.177085, and 0.426102. The script also sets session-wide default graphics formatting and plots the wealth-tail probability.

3 Python

The Python implementation is an installable NumPy/SciPy package in the `python` folder. It requires Python 3.10 or later. From that folder, install the package and optional example and test dependencies with

```
python -m pip install -e ".[example,test]"
pytest
python examples/example.py
```

The public functions use snake-case names and are imported from the package as follows:

```
from pareto_extrapolation import (
    exp_grid,
    get_q,
    get_top_shares,
    get_zeta,
)
```

The state-pair and joint-state orderings are identical to the MATLAB version. Python arrays are zero-indexed, but users do not need to transform the data ordering. One-dimensional arrays replace MATLAB row or column vectors.

3.1 Exponential grid: `exp_grid`

```
grid = exp_grid(a, b, c, n)
```

This is the direct NumPy counterpart of `expGrid.m`. It returns a one-dimensional array containing both endpoints exactly and applies the same input restrictions and shifted-log formula.

3.2 Pareto exponent: `get_zeta`

```
zeta, type_dist = get_zeta(ps, pj, v, g, zeta_bound=(1e-2, 100.0))
```

The array shapes and probability conventions match `getZeta.m`. A one-dimensional `pj` is interpreted as a state-independent shock distribution. The implementation uses dense NumPy eigenvalue calculations for the small $S \times S$ matrix and SciPy's bracketed root solver. It issues `RuntimeWarning` and returns an empty NumPy array when the exponent lies outside the search interval.

3.3 Joint transition matrix: `get_q`

```
q, pi_star = get_q(
    ps, pj, v, x0, x_grid, gstjn,
    g_stj=None, zeta=None,
    compute_stationary=True,
)
```

The input meanings and array order match `getQ.m`. The function returns `q` as a SciPy CSR sparse matrix and `pi_star` as a one-dimensional NumPy array. To skip the stationary calculation, use

```
q, pi_star = get_q(..., compute_stationary=False)
```

The second returned value is then `None`. If `g_stj` or `zeta` is `None`, it is estimated or computed internally.

The marginal grid distribution is

```
x_dist = pi_star.reshape(S, N).sum(axis=0)
```

Unlike the legacy-compatible MATLAB interface, the Python function raises `ValueError` when $\zeta \leq 1$ instead of returning a NaN matrix.

3.4 Top wealth shares: `get_top_shares`

```
top_share = get_top_shares(top_prob, w_grid, w_dist, zeta=0.0)
```

This function follows the revised negative-wealth behavior of `getTopShares.m` and uses SciPy’s shape-preserving PCHIP interpolator. A scalar probability input returns a float. A one-dimensional input returns a NumPy array. The MATLAB and SciPy PCHIP implementations can differ at about the 10^{-10} level even when their interpolation knots agree.

3.5 Example and tests

The script `python/examples/example.py` reproduces the MATLAB example, prints the exponent, type distribution, top shares, and sparse-matrix size, and plots the wealth tail with Matplotlib.

The tests in `python/tests` cover endpoint construction, exact-bound roots, invalid inputs, negative wealth, zero mass at the largest grid point, the complete economic example, and a fixed transition matrix generated by the MATLAB implementation. The transition entries agree to floating-point precision; the end-to-end top shares agree within 10^{-10} .

References

- [1] Brendan K. Beare and Alexis Akira Toda. “Determination of Pareto Exponents in Economic Models Driven by Markov Multiplicative Processes.” *Econometrica*, 90(4):1811–1833, 2022. [doi:10.3982/ECTA17984](https://doi.org/10.3982/ECTA17984).
- [2] Émilien Gouin-Bonenfant and Alexis Akira Toda. “Pareto Extrapolation: An Analytical Framework for Studying Tail Inequality.” *Quantitative Economics*, 14(1):201–233, 2023. [doi:10.3982/QE1817](https://doi.org/10.3982/QE1817).